



Arquitectura de Análisis y Trading Predictivo en Mercados de Futuros de Criptomonedas

Trabajo Fin de Máster

Máster en Big Data & Data Engineering

Víctor Daniel Rodríguez Pérez

Tutores: Jorge Centeno y Alberto González

18 de septiembre de 2025

1. Resumen 6
2. Palabras clave 7
3. Introducción 7
3.1. Contextualización del proyecto 7
3.2. Objetivos del proyecto 7
3.3. Justificación e interés 8
4. Metodología 9
4.1. Diseño general del proyecto 9
4.2. Proceso de trabajo y fases (planificación) 9
5. Arquitectura 10
5.1. Diagrama general 10
5.2. Descripción de la arquitectura técnica (versión extendida en Anexo 8.6). 10
5.3. Justificación de elección de tecnologías 10
Azure Databricks 10
Databricks Jobs 11
Apache Spark 11
Azure Data Lake Storage Gen2 11
Unity Catalog 11
MLflow 12
Docker 12
Azure Container Instances (ACI) 12
Python 12
API REST + WebSocket (Binance Futures) 13
5.4. Estimación básica de costes de infraestructura 13
5.5. Estrategia DevOps 14
6. Solución tecnológica 16

6.1. Fuentes de datos (se amplía en anexo 8.4)....16
6.2. Orquestación y flujos de datos: preparación y procesamiento....16
Databricks Jobs (capturas en el anexo apartado 8.11)....16
6.3. Almacenamiento y consulta....18
6.4. Explotación de resultados....18
Modelos....18
Reporting....19
6.5. Consideraciones legales y éticos del uso de datos....20
7. Resultados y conclusiones....21
7.1. Logros alcanzados....21
7.2. Métricas utilizadas y resultados obtenidos....21
7.3. Limitaciones identificadas....22
7.4. Futuras líneas de mejora y/o desarrollo....23
Mejora del modelo....23
Generación de whl....23
Creación de un flujo completamente automatizado....23
Desarrollo de paneles de visualización y reporting....24
8. Bibliografía....25
9. Anexos....27
9.1. Analisis de costos de despliegue a producción....27
9.2. Cálculo del percentil 95....28
9.3. Análisis comparativo de latencias por características de servidores....28
9.4. Fuentes de datos en profundidad....29
9.4.1. Fuentes de datos API REST....29
Binance:....29
Binance SPOT....29

Bybit....31
OKX....31
CoinGecko....32
Agregador de datos independiente de criptomonedas más grande del mundo. No permite realizar operaciones, solo provee información....32
9.4.2. Fuentes de datos Websocket....33
Binance Websocket....33
9.5. Proceso de trabajo y fases (ampliación)....33
Fase 1: Evaluación y elección de los datos necesarios para analizar las criptomonedas candidatas a estudio en profundidad....33
Fase 2: Análisis de fuentes de datos....34
Fase 3: Evaluación y elección de tecnologías....34
Fase 4: Ingesta y almacenamiento de datos....35
Fase 5: Limpieza y análisis de los datos....35
Fase 6: Elección de la criptomoneda más interesante....35
Fase 7: Ingesta y almacenamiento de datos de las criptomonedas elegidas..36
Fase 8: limpieza y análisis de los datos....36
Fase 9: construcción del prototipo de trading en tiempo real....36
Fase 10: análisis de los datos en base a la latencia obtenida....37
9.6. Descripción de la arquitectura técnica (ampliación)....37
9.7. Diagrama General (archivo de imagen adjunto en el entregable)....38
9.8. Creación y desempeño de contenedor Azure Container Instances....39
9.9. Relación de tablas creadas en Unity Catalog....42
BRONZE....42
SILVER....43
GOLD....45
9.10. MLflow + Unity Catalog....46
9.11. Databricks Jobs....48

9.12. Docker Hub.....52
8.13. Azure Data Factory.....53
8.14 Azure Container Instances.....54

1. Resumen

Se diseñó e implementó un pipeline de datos de extremo a extremo para analizar la variabilidad del precio de las criptomonedas listadas en Binance Futures y estimar la probabilidad de revalorización tras compras de volumen inusualmente alto (“whale trades”) en distintos horizontes temporales.

Se ingirieron datos abiertos desde varias fuentes mediante Azure Data Factory y Azure Databricks; se procesaron en Azure Databricks y se almacenaron en Azure Data Lake Storage (ADLS Gen2) en formato Delta Lake, siguiendo arquitectura Medallion (raw, bronze, silver y gold).

En una primera fase se seleccionaron los activos a estudiar conforme a criterios objetivos, entre ellos: actividad suficiente (volumen y liquidez) e índice de impacto de precio por unidad de flujo de órdenes (índice de Kyle, λ), priorizando aquellos cuya cotización pudiera ser a priori más susceptible a estos eventos.

En la fase final, se creó un Job de Databricks que recoge la criptomoneda más susceptible atendiendo a los criterios anteriores, se recabaron datos en detalle de cotizaciones por segundo durante los últimos 180 días de este activo y se procesaron para alimentar el entrenamiento de un modelo de clasificación binaria de regresión logística que emitiera señales predictivas de si una operación de compra de esa criptomoneda, produciría un incremente en la cotización de la misma en diferentes horizontes temporales.

Los resultados obtenidos confirman la viabilidad técnica de una solución de alertas y apoyo a la decisión casi en tiempo real y sientan las bases para su explotación operativa.

2. Palabras clave

- Ingeniería de Datos
- Binance Futures
- Whale trades
- Microsoft Azure
- Azure Databricks
- Arquitectura Medallion
- Regresión Logística

3. Introducción

3.1. Contextualización del proyecto

El mercado de criptomonedas ha experimentado un crecimiento exponencial en la última década, acompañado por una volatilidad excepcionalmente alta. Este escenario atrae tanto a inversores tradicionales como institucionales, generando un interés creciente en analizar profundamente los patrones que subyacen a los movimientos de mercado. Las blockchain públicas facilitan el acceso abierto y continuo a enormes volúmenes de datos transaccionales y financieros, convirtiendo al ecosistema cripto en un entorno idóneo para aplicar técnicas avanzadas de ingeniería de datos. En este contexto, resulta clave la capacidad de extraer, almacenar y procesar información tanto en batch como en tiempo real, identificando oportunidades anticipadas mediante la detección precisa de eventos críticos que puedan afectar significativamente los precios.

3.2. Objetivos del proyecto

El presente Trabajo Fin de Máster tiene como objetivos principales:

- **Objetivo 1:** Identificar mediante análisis histórico qué criptomonedas presentan, a priori,

una mayor sensibilidad a grandes operaciones de compra.

- **Objetivo 2:** Diseñar e implementar un pipeline completo de ingeniería de datos capaz de extraer de forma eficiente por una parte datos históricos en formato batch y por otra, datos en tiempo real (on-chain y off-chain) con alta frecuencia. La frecuencia se establecerá en función del coste.

- **Objetivo 3:** Intentar establecer una o varias correlación/es entre las grandes transacciones o *whale trades* y movimientos significativos del precio, evaluando la causalidad y la solidez temporal de dicha relación.

- **Objetivo 4:** Creación de un prototipo funcional que detecte estos eventos críticos, generando señales predictivas con la mínima latencia y máxima fiabilidad posible, facilitando decisiones de inversión estratégicas y oportunas.

- **Objetivo 5:** Despliegue de este prototipo para predecir señales en tiempo real, intentando minimizar latencias acumulativas.

3.3. Justificación e interés

La elección del ámbito cripto para este trabajo está justificada por la disponibilidad masiva de datos reales, la relevancia económica del fenómeno estudiado (impacto de grandes operaciones sobre el mercado) y la complejidad técnica que implica procesar información altamente volátil y diversa en tiempo real.

Desde el punto de vista académico, este proyecto permite aplicar los conocimientos adquiridos en el Máster en Big Data e Ingeniería de Datos de manera integrada y práctica, demostrando competencias avanzadas en arquitecturas de datos escalables, streaming analytics, almacenamiento eficiente y generación automática de señales predictivas.

Además, desde un punto de vista profesional y económico, el desarrollo de un prototipo capaz de anticipar movimientos de mercado presenta un valor evidente, tanto en el ámbito personal como en posibles aplicaciones futuras en el sector fintech y financiero en general.

4. Metodología

4.1. Diseño general del proyecto

El diseño general del proyecto se ha basado en una arquitectura modular y escalable siguiendo un enfoque iterativo, incremental, con ciclos de desarrollo que han permitido validar (o no), componentes por separado antes de integrarlos en una solución coherente y funcional. Motivado por el carácter experimental del proyecto, notebooks de Databricks ha sido la principal herramienta de desarrollo, así como de ejecución en conjunto con Databricks Jobs y Azure Data Factory.

4.2. Proceso de trabajo y fases (planificación)

El proyecto ha sido llevado a cabo siguiendo las siguientes fases (ampliado en anexo 8.5)

- Fase 1: Evaluación y elección de los datos necesarios para analizar las criptomonedas candidatas a estudio en profundidad
- Fase 2: Análisis de fuentes de datos
- Fase 3: Evaluación y elección de tecnologías
- Fase 4: Ingesta y almacenamiento de datos
- Fase 5: Limpieza y análisis de los datos
- Fase 6: Elección de la criptomoneda más interesante
- Fase 7: Ingesta y almacenamiento de datos de las criptomonedas elegidas
- Fase 8: Limpieza y análisis de los datos
- Fase 9: Desarrollo del prototipo de trading en tiempo real
- Fase 10: Análisis de los datos en base a la latencia obtenida

5. Arquitectura

5.1. Diagrama general

Anexo 8.7 y adjunto en el entregable

5.2. Descripción de la arquitectura técnica (versión extendida en Anexo 8.6)

- Lakehouse en ADLS Gen2 con arquitectura Medallion (raw→bronze→silver→gold).
- Ingesta de datos recopilados de APIs de exchanges vía Spark y Azure Data Factory.
- Unity Catalog gestiona catálogos, esquemas y *lineage*.
- Procesamiento Databricks orquestado en jobs.
- MlFlow registro de los distintos modelos entrenados, sus parámetros, sus resultados y la versión del código que lo entrenó.
- Contenedor en Azure Containers Instances para despliegue una imagen publicada en Docker Hub, conteniendo script Python que descarga el modelo desde MLflow, conecta con el websocket de Binance y realiza predicciones en tiempo real, midiendo también latencias y retrasos.

5.3. Justificación de elección de tecnologías

La elección de tecnologías utilizadas en el desarrollo de este proyecto se ha basado en criterios de flexibilidad, integración nativa en cloud, compatibilidad con procesamiento batch y streaming. Los componentes elegidos han sido los siguientes:

Azure Databricks

Se ha elegido como entorno principal de desarrollo y ejecución por ser una plataforma cloud que integra Apache Spark, Delta Lake y MLflow en un solo servicio. Ofrece clústeres escalables bajo demanda, soporte nativo para procesamiento batch y streaming, ejecución de notebooks, tracking de modelos, y control de acceso avanzado mediante Unity Catalog, lo que facilita un desarrollo rápido, reproducible y gobernado.

Azure Data Factory

Se ha utilizado como herramienta principal de orquestacion, ya que auna un gran número de características que lo hacen interesante: integración nativa con el resto de servicios de Azure (nos interesa principalmente por Databricks Jobs), permite automatización de tareas recurrentes, está desacoplado del entorno de ejecución y por su sencillez de monitorización

Databricks Jobs

Se ha utilizado como orquestador secundario de todas las actividades en Databricks una vez desarrolladas. Es parte de Databricks y nos sirve para llevar a cabo el proceso de extracción de datos y entrenamiento del modelo de manera automatizada y periódica.

Apache Spark

Motor de procesamiento distribuido utilizado en todas las etapas del pipeline. Permite manejar grandes volúmenes de datos, ejecutar transformaciones complejas en paralelo y procesar flujos tanto en tiempo real como en modo batch. Es el núcleo del procesamiento dentro de Databricks.

Azure Data Lake Storage Gen2

Se ha utilizado Azure Data Lake Storage Gen2 (ADLS Gen2) como sistema de almacenamiento físico en la nube, Delta Lake como formato de almacenamiento transaccional sobre dicho sistema y arquitectura tipo Medallion(raw, bronze, silver, gold) para gestionar las transformaciones de los datos. Esta combinación permite almacenar grandes volúmenes de datos de forma escalable y eficiente, al tiempo que se incorporan funcionalidades avanzadas como transacciones ACID, control de versiones (time travel), evolución de esquemas, y soporte nativo para operaciones MERGE, UPDATE y DELETE.

Unity Catalog

Se ha elegido Unity Catalog como sistema de gestión de datos por su capacidad para proporcionar trazabilidad, control de acceso unificado, versionado de modelos y organización estructurada de los activos de datos dentro de la plataforma Databricks.

Su uso ha permitido registrar todas las tablas del pipeline (bronze, silver, gold) de forma no gestionada, facilitando la navegación entre capas, la consulta mediante SQL y la auditoría de cambios o accesos. Unity Catalog también ofrece auditoría centralizada de accesos, control de versiones a nivel de metadatos y una vista global del sistema de datos, lo que refuerza la integridad y reproducibilidad del proyecto. Su integración directa con MLflow añade un valor adicional, al permitir relacionar y almacenar fácilmente datasets, modelos y experimentos dentro del mismo entorno gobernado.

MLflow

Se ha usado para gestionar el ciclo completo de los modelos de machine learning: tracking de experimentos, registro de métricas, artefactos y parámetros, y versionado de modelos. Su integración en Databricks permite un flujo continuo desde el entrenamiento hasta el deployment o análisis.

Docker

Docker se ha utilizado para contenerizar scripts clave y facilitar su despliegue en entornos controlados, como Azure Container Instances. Esta portabilidad asegura que las dependencias y configuraciones del entorno sean consistentes, lo que mejora la reproducibilidad y permite validar componentes del sistema fuera de Databricks.

Azure Container Instances (ACI)

ACI se ha utilizado como solución ligera y económica para desplegar y ejecutar scripts de inferencia en tiempo real desde contenedores Docker, especialmente aquellos que consumen datos por WebSocket (por ejemplo, desde Binance Futures) y aplican modelos registrados en MLflow. Esta solución permite mantener procesos activos 24/7 sin necesidad de gestionar infraestructura compleja, lo cual es ideal en el contexto académico del proyecto.

Python

Lenguaje principal del proyecto por su versatilidad, su ecosistema rico en librerías científicas y de integración (como pandas, requests, ujson, websockets) y su compatibilidad total con PySpark, MLflow y Docker. Además, su facilidad de uso ha permitido iterar rápidamente durante el desarrollo.

API REST + WebSocket (Binance Futures)

Se ha optado por consumir datos directamente desde la API pública de Binance para obtener información de mercado tanto histórica como en tiempo real. La API REST permite acceder a velas, funding rates y open interest, mientras que WebSocket proporciona flujos en vivo de operaciones agregadas (aggTrades) y precios de referencia (markPrice). Esta dualidad ha sido clave para entrenar modelos con contexto y generar una herramienta que haga inferencia en tiempo real.

5.4. Estimación básica de costes de infraestructura

El desarrollo ha sido llevado a cabo usando notebooks en Azure Databricks, almacenamiento en Azure Data Lake Storage Gen2. Todos los componentes, salvo Azure Container Instances que ha sido desplegado en Japan East, han sido desplegados en Central India ya que es la región con menor coste y para desarrollo, las latencias no nos eran importantes. El grueso del desarrollo ha sido realizado en 10 días intensivos, por lo tanto los cálculos de costes han sido realizados sobre un marco temporal de 10 días.

Producto	Uso	Coste
Azure Databricks D3 v2: 4 vCPU, 14 GB RAM single node	10 días	80,88
Almacenamiento ADLS Gen2	30 Gb	6
Azure Data Factory	10	5
Azure Container Instances	1 día	1.34€
Total		**93,22€**

Una vez realizado el grueso del desarrollo, actualizar los datos para la selección de las criptomonedas candidatas a estudio en profundidad, ingestar los datos para el entrenamiento del modelo y el entrenamiento en sí, tendrá los siguientes costes aproximados:

Producto	Uso	Coste
Azure Databricks	2h	2€

D3 v2: 4 vCPU, 14 GB RAM single node		
Storage ADLS Gen2	2 Gb	0,5€
Azure Data Factory	1h	1€
Azure Container Instances	1 día	1.34€
Total		**4,84€**

Hay que considerar que solamente por mantener la arquitectura en la nube y el almacenamiento de datos en ADLS Gen2 , aunque económico, tiene un coste. Así que el coste de reentrenamiento será más elevado.

5.5. Estrategia DevOps

Databricks: versionado notebooks

Al estar trabajando sobre un flujo experimental y muy dinámico, se ha utilizado Databricks Notebooks como principal herramienta de desarrollo, contando ya con un control de versiones embebido que ha sido el que se ha utilizado

MLflow

Ha utilizado como herramienta central de tracking de experimentos durante el desarrollo del modelo de predicción de movimientos significativos en criptomonedas. Esta herramienta ha permitido registrar de forma automática y organizada todos los hiperparámetros, métricas, artefactos y modelos entrenados en las distintas fases del proceso de validación.

Adicionalmente, se ha utilizado para exportar el mejor modelo identificado, el cual ha sido posteriormente desplegado en un contenedor en Azure Container Instances para realizar predicciones en tiempo real, cerrando así el ciclo completo desde la experimentación hasta la producción.

GitHub

En las últimas fases del proyecto, se ha enlazado el entorno de trabajo en Azure Databricks con un repositorio privado de GitHub para garantizar el versionado del código fuente y facilitar la colaboración y trazabilidad de los cambios. Esta integración ha permitido sincronizar

notebooks y scripts directamente con el repositorio, manteniendo una copia centralizada y actualizada del código del proyecto separada del entorno Azure.

Azure Data Factory

En este producto de Azure es el principal orquestador del proyecto, como se detalla en la sección *6.4 Orquestación.*

Databricks Jobs

Esta funcionalidad de Databricks ha sido utilizada como orquestador secundario como se detalla en la sección *6.4 Orquestación.*

6. Solución tecnológica

6.1. Fuentes de datos (se amplía en anexo 8.4)

Binance API REST: Extraemos metadatos de mercado (símbolos, filtros de precio y cantidad, precisiones, monedas base/cotizada) y series históricas para modelado: velas OHLCV por intervalo, operaciones agregadas (precio, cantidad, dirección del agresor), y en derivados mark/index price, funding y open interest. Los precios y cantidades llegan como cadenas decimales y los tiempos en milisegundos.

Binance Websocket: Extraemos flujo en tiempo real de: trades agregados con precio, cantidad y bandera del agresor, y top of book (mejor bid/ask y cantidades). Con ello calculamos mid price, spread, desequilibrios del libro, ventanas de volumen 1–5 s y retornos por segundo, además de latencias (red/procesado).

Bybit API REST: Extraemos catálogos de instrumentos de spot y derivados con sus parámetros de negociación: símbolo, monedas base/cotizada/liquidación, precisiones, tick size, pasos y mínimos de cantidad, y en derivados límites de apalancamiento y ciclos de funding. Esto permite normalizar reglas entre mercados y validar tamaños/umbrales.

OKX API REST: Extraemos el inventario de instrumentos de spot y swaps perpetuos con su identificación, monedas, tamaños/precisiones (tick, lot, mínimos) y, en derivados, detalles de contrato (valor/multiplicador, moneda de liquidación, subyacente). Se usa para armonizar filtros y verificar parámetros frente a otras fuentes.

CoinGecko API REST: Extraemos metadatos no transaccionales por activo: nombre, símbolo, descripciones, enlaces oficiales, contratos por cadena, imágenes y, cuando se requiere contexto, resúmenes de mercado agregados. Sirve para enriquecer la documentación y la trazabilidad del activo de forma independiente a un exchange concreto.

6.2. Orquestación y flujos de datos: preparación y procesamiento

Se han implementado 2 flujos de datos principales:

Databricks Jobs (capturas en el anexo apartado 8.11)

- whale_trades-calculate_scores: recoge las criptomonedas disponibles en Binance y descarta

las que no se ajustan a nuestro propósito según los siguientes criterios:

- menos de 80% de días con valores validos
- más de 3 días consecutivos sin información
- más de 2% días sin datos
- descarta stable coins
- volumen inferior a 1.00.000 USDT
- media de trades diarios inferior a 2.000 en los últimos 90 días
- no están en estado 'trading'

Una vez descartados, se asigna un score final que es la suma ponderada entre:

Criterio	Ponderación
Índice de Kyle, λ	0,6
Proporción de días en los que el rango entre el máximo y el mínimo superó el 20% del precio de apertura	0,25
Proporción de días en los que el precio varió al menos un 5% entre apertura y cierre	0,15

- whale_trades-extract_train_model: en base al score anterior, elige la criptomoneda que a priori tendrá más variabilidad por grandes operaciones y ejecuta consultas sobre la API de futuros de Binance de los últimos 180 días de operaciones, agrupadas con una periodicidad de 1 segundo. Tras ello, opera sobre los datos para rellenar los huecos sin transacciones, contruye features y labels para posteriormente entrenar, evaluar y registrar el modelo en MLflow. Posteriormente se reempaqueta el pipeline a pyfunc y se registra en Unity Catalog con la intención de poder ser descargado para su

uso. Tras numerosos experimentos, los parámetros usados son los siguientes:

Parámetro	Valor
min_signals	200
precision_target	0.7
regParam_best	0.02
target_horizon_sec	21600 (6h)
threshold_policy	fallback

6.3. Almacenamiento y consulta

Los datos son almacenados en Azure Data Lake Storage Gen2, siguiendo una arquitectura Medallion y consultados por los distintos notebooks de Databricks para ser procesada y posteriormente servir para entrenar el modelo predictivo final. A la vez que se almacenan esos datos, se han registrado sus ubicaciones como tablas no gestionadas de Unity Catalog, teniendo así un gobierno centralizado de los, proporcionando trazabilidad y linage y permitiéndonos hacer consultas en SQL, con la flexibilidad que ello proporciona.

6.4. Explotación de resultados

Modelos

Desde la definición del proyecto, se definió como objetivo central la construcción de un sistema capaz de realizar predicciones sobre potenciales subidas en la cotización de ciertas criptomonedas, aprovechando datos de alta frecuencia y eventos de volumen anómalamente grandes. Para ello, se ha desarrollado e integrado un módulo de entrenamiento y evaluación de modelos supervisados, directamente conectado a la arquitectura de datos diseñada.

Los modelos se entrenan a partir de datos ingestados, procesados y almacenados en ADLS Gen2. A través de un sistema de orquestación automatizado, los datos relevantes se seleccionan, transforman y particionan por símbolo, permitiendo así el entrenamiento de

dichos modelos

Durante el proceso de experimentación se registran automáticamente todas las métricas, hiperparámetros, artefactos y modelos mediante MLflow, lo que permite versionar y comparar modelos de forma controlada. Se aplican métricas específicas para entornos con desbalanceo de clases (como el Area Under the Precision-Recall Curve), así como análisis de retorno asociado a las señales generadas.

Los modelos se entrenan con una ventana histórica amplia (hasta 90 días). Una vez decidido cuál es el mejor modelo actual, se añade la información necesaria al contenedor de Azure Container Instances para su despliegue en la región Japan East de Azure para comenzar la inferencia en tiempo real, siendo capaz de recibir datos desde el websocket de Binance y emitir señales predictivas basadas en inputs recientes.

Toda esta lógica de modelado ha sido desarrollada como un módulo dentro del pipeline general, facilitando su actualización periódica con nuevos datos, y su eventual despliegue en producción sobre contenedores ligeros. La arquitectura contempla tanto la escalabilidad como la trazabilidad de cada experimento.

Reporting

Una vez desplegada la arquitectura de ingestión y procesamiento, todos los datos generados —incluyendo eventos individuales de mercado, agregaciones a nivel temporal, indicadores calculados y etiquetas— se encuentran estructurados y disponibles en formato Delta Lake dentro de Azure Data Lake Storage Gen2, organizados por capas (bronze, silver y gold) y catalogados mediante Unity Catalog.

Esta disponibilidad garantiza que cualquier análisis posterior, visualización o integración con herramientas de reporting como Power BI, Tableau o notebooks exploratorios puede realizarse de forma directa, segura y escalable. La información puede ser consultada desde distintos niveles de granularidad y por cualquier dimensión relevante (símbolo, día, tipo de evento, etc.).

Sin embargo, dado que el objetivo principal del presente trabajo ha sido el diseño e implementación de un modelo predictivo, no se ha profundizado en el desarrollo de informes operativos, cuadros de mando u otras formas de reporting visual o analítico. Se deja, por tanto, como trabajo futuro el diseño de paneles que permitan explorar la actividad de mercado, la

evolución de los indicadores y el impacto de los eventos relevantes sobre los precios.

6.5. Consideraciones legales y éticos del uso de datos

Fuente y alcance de los datos extraídos de Binance API

El proyecto consume datos públicos de mercado (precios, trades y profundidad) a través de los endpoints oficiales de la API de Binance (REST/WebSocket), cumpliendo rate limits y guías técnicas.

Base legal de uso extraídos de Binance API

Binance no ofrece una licencia abierta de datos; el uso se rige por sus Términos de Uso, que conceden una licencia no exclusiva sobre la "Binance IP" únicamente en la medida necesaria para recibir el servicio y para uso personal o interno no comercial. Este TFM utiliza los datos exclusivamente con fines académicos y sin explotación comercial.

Límites y prohibiciones relevantes extraídos de Binance API

Se prohíbe copiar/almacenar/transferir/publicar/licenciar la Binance IP o crear obras derivadas fuera del uso permitido; también se prohíben crawlers/scraping no autorizados y cualquier intento de eludir restricciones técnicas. El proyecto no redistribuye datos brutos ni expone feeds, solo presenta resultados agregados (métricas/figuras) que no sustituyen el servicio original.

Seguridad y privacidad

No se tratan datos personales (solo market data). Las API keys (si se usaran para endpoints de cuenta) se custodian siguiendo buenas prácticas (mínimos permisos, rotación, almacenamiento seguro).

7. Resultados y conclusiones

7.1. Logros alcanzados

Reducción de la latencia de predicción

Al desplegar el código Python que conecta con el websocket de Binance del que recibe las operaciones realizadas y realiza las predicciones con el modelo entrenado en Databricks en la región de Azure Japan East, hemos conseguido reducir en más de 150ms la latencia, como se puede ver en la siguiente tabla:

Concepto	Local (Canarias)	Japan East	Reducción de latencia media	Diferencia (%)
Latencia en recibir el mensaje	268,7 ms	109,5 ms	159,2 ms	-0,59
Tiempo de procesado	6,9 ms	5.0 ms	1,9 ms	-0,28
Total	**275,8 ms**	**112,7 ms**	**163,1 ms**	**-59.1%**

Reducción de los costes de mantener en producción

Al utilizar Azure Container Instances como servicio para tener en producción nuestra herramienta, hemos conseguido reducir un 54,12% los costes fijos con respecto al siguiente producto de Azure más económico (Virtual Machines).

7.2. Métricas utilizadas y resultados obtenidos

Se ha seguido un proceso de prueba y error para tratar de converger a un modelo con un desempeño lo mejor posible, el cual obtiene las siguientes métricas:

val_avg_ret_pred_at_best_t (0.00808 ≈ 0,81%)

Mide el retorno medio bruto por señal en validación al umbral óptimo. Resume el valor

económico esperado de cada trade.

ret_p50 (0.00932 ≈ 0,93%)

Mide la mediana del retorno por señal (resultado "típico", menos sesgo por colas).

ret_percentiles (p10, p25, p75, p90 = −0.81%, 0.00%, 1.72%, 2.37%)

Miden la dispersión y asimetría; delimitan pérdidas típicas y ganancias altas.

avg_ret_hits (1.87%)

Mide el payoff medio cuando la señal acierta (label=1).

avg_ret_misses (−0.23%)

Mide el coste medio de un fallo (label=0).

share_ret_gt_0 (73.20%)

Mide el % de operaciones ganadoras (>0).

share_ret_ge_1pct (49.48%)

Mide % de señales que superan el objetivo (≥1%).

val_precision_pos (49.48%)

Mide la calidad de las señales (menos falsos positivos).

val_signals (291)

Mide la frecuencia de operaciones.

best_threshold (0.50859)

Mide el punto operativo que equilibra cantidad vs. calidad según tu criterio de validación.

7.3. Limitaciones identificadas

Riesgo de slippage tras whale-trades

El *slippage* es la diferencia entre el precio al que planificas entrar/salir y el precio real de ejecución.Al reducirse la liquidez tras una gran operación de compra, esa diferencia puede ser significativa. Nuetro pipeline evalúa señales sobre cotizaciones teóricos sin simular ejecución real; por tanto, los retornos estimados podrían estar inflados.

Para evitar este problema, conociendo de antemano que cada exchange tiene su mercado de futuros independiente, se plantea lanzar una operación de compra en otro exchange. Para ello hay que tener en cuenta que existen bots de arbitraje que harán que el precio se iguale con una latencia muy pequeña, así que el objetivo sería mejorar esa latencia y así podríamos evitar esta limitación.

7.4. Futuras líneas de mejora y/o desarrollo

Mejora del modelo

Tras la experimentación llevada a cabo en Databricks, se generó un contenedor docker conteniendo un script de Python que descarga el modelo, se conecta al websocket público de Binance, haciendo predicciones en tiempo real sobre las transacciones sobre las criptomonedas elegidas para el análisis. Al observar el desempeño de dicho contenedor, observamos que las predicciones se realizan en 6ms (de media) por lo que se propone enriquecer el modelo con mayor cantidad de datos para mejorar la capacidad de predicción y las métricas de acierto del mismo.

Generación de whl

Una vez se alcance una versión de la arquitectura estable, se debe generar un paquete whl para facilitar la reproducibilidad de la herramienta. No se ha hecho durante la elaboración del presente TFM por falta de tiempo.

Creación de un flujo completamente automatizado

Se propone, mediante el uso de SDK de Azure para Python,integrar en el proceso la creación y el despliegue del contenedor que ejecuta el script de Python que conecta con el websocket de Binance y ejecuta las consultas al modelo. La versión actual va desde la extracción de los datos hasta el entrenamiento y registro del modelo, pasando por la limpieza y modelado de los datos.

Borrado y archivado de datos generados

El proceso de desarrollo del presente TFM ha sido un proceso iterativo en el cual se ha ido recabando información sobre el mercado de criptomonedas, la manera de trabajar del mismo y métricas asociadas a ello. Se ha generado gran cantidad de información, almacenada en ADLS Gen 2 y registrada en Unity Catalog, mucha de la cual no es necesaria en el momento actual. Por ello, se debe evaluar la necesidad de cada tabla y hacer un borrado o archivado de todas ellas que no vayan a ser necesarias en el corto plazo o no vayan a ser necesarias en absoluto.

Desarrollo de paneles de visualización y reporting

Se propone la implementación de paneles de visualización orientados a facilitar la exploración de los datos procesados por el sistema. Dado que actualmente existen múltiples herramientas públicas y gratuitas que ofrecen visualizaciones estándar del mercado, el desarrollo debe centrarse en aportar valor diferencial mediante la exposición de información que no está disponible de forma abierta. Gracias a que los datos ya se encuentran estructurados, particionados y gobernados en Delta Lake mediante Unity Catalog, este tipo de paneles podrían implementarse fácilmente utilizando herramientas como Power BI o los propios notebooks de Databricks, aportando una capa adicional de explotación orientada a la toma de decisiones.

8. Bibliografía

- Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Ghodsi, A., … Zaharia, M. (2019). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. arXiv:1906.01969. https://arxiv.org/abs/1906.01969
- Apache Software Foundation. (s. f.). Apache Spark — Structured Streaming Programming Guide. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- Apache Software Foundation. (s. f.). Apache Spark — SQL, DataFrames and Datasets Guide. https://spark.apache.org/docs/latest/sql-programming-guide.html
- Apache Software Foundation. (s. f.). Apache Spark — Window Functions. https://spark.apache.org/docs/latest/sql-ref-functions-builtin.html#window-aggregate-functions
- Binance. (s. f.). Binance API (Futures) — REST & WebSocket. https://binance-docs.github.io/apidocs/futures/en/
- Binance. (s. f.). Términos de uso (Terms of Use). https://www.binance.com/en/terms
- Bybit. (s. f.). Bybit API v5 Documentation. https://bybit-exchange.github.io/docs/v5/intro
- CoinGecko. (s. f.). CoinGecko API v3 Documentation. https://www.coingecko.com/en/api/documentation
- Databricks. (s. f.). Arquitectura Lakehouse y modelo Medallion. https://docs.databricks.com/en/lakehouse/medallion.html
- Databricks. (s. f.). Unity Catalog (gobernanza de datos y AI). https://docs.databricks.com/en/data-governance/unity-catalog/index.html
- Databricks. (s. f.). Workflows & Jobs (orquestación en Databricks). https://docs.databricks.com/en/workflows/jobs/index.html
- Delta Lake. (s. f.). Documentación oficial de Delta Lake. https://docs.delta.io/latest/
- Docker. (s. f.). Docker Documentation. https://docs.docker.com/

- IETF. (2011). RFC 6455 — The WebSocket Protocol. https://datatracker.ietf.org/doc/html/rfc6455
- Kyle, A. S. (1985). Continuous Auctions and Insider Trading. Econometrica, 53(6), 1315–1335. https://doi.org/10.2307/1913210
- Microsoft. (s. f.). Azure Databricks — Documentación oficial. https://learn.microsoft.com/azure/databricks/
- Microsoft. (s. f.). Azure Data Factory — Introducción. https://learn.microsoft.com/azure/data-factory/introduction
- Microsoft. (s. f.). Azure Data Lake Storage Gen2 — Introducción. https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction
- Microsoft. (s. f.). Azure Container Instances — Información general. https://learn.microsoft.com/azure/container-instances/container-instances-overview
- MLflow. (s. f.). MLflow Documentation. https://mlflow.org/docs/latest/index.html
- Meng, X., Bradley, J., Yuvaz, B., Sparks, E., Venkataraman, S., Liu, D., … Zaharia, M. (2016). MLlib: Machine Learning in Apache Spark. Journal of Machine Learning Research, 17(34), 1–7. https://jmlr.org/papers/v17/15-237.html
- OKX. (s. f.). OKX API (v5) — Public Data. https://www.okx.com/docs-v5/en/
- Python Software Foundation. (s. f.). Python 3 Documentation. https://docs.python.org/3/
- Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot when Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression (3.ª ed.). Wiley.

9. Anexos

9.1. Analisis de costos de despliegue a producción

A la hora de desplegar el websocket con el que obtendremos la información directamente de Binance Futures y ejecutar el modelo previamente entrenado, necesitamos un entorno para ello. Se contemplaron los siguientes servicios:

- Virtual Machine

- Azure Containers App,

- Azure Kunermetes Service (AKS)

- Containers Instances

Los recursos mínimos estimados para la ejecución de ambos componentes son:

- 1Gb Memoria

- 1 vCPU

Se ha elegido la región Japan East para el despliegue ya que es la región más próxima geográficamente a los servidores de Binance. El coste de mantener los servicios activos **24/7** en dicha región son los siguientes (24h):

- Virtual Machine: 1,94€

- Azure Container Apps (mínimo 16Gb RAM y 4 vCPU): 7,39€

- Container Instances: 0,89€

- Azure Kunernetes Service (AKS) (mínimo 4Gb RAM y 2vCPU): 2,71€

Por sencillez y precio, se elige **Azure Container Instances** para llevar a cabo el despliegue del modelo y la conexión websocket con el servidor de Binance desde Japan East.

9.2. Cálculo del percentil 95

```
from pyspark.sql import functions as F

df = spark.sql("SELECT * FROM dbrtfmprod.whale_trades.bronze__top_kyles_all_trades")
df = df.withColumn('amount', F.col("price")*F.col("qty"))
display(
        df_full.groupBy("symbol")
                .agg(
                        F.round(
                                F.expr("percentile_approx(amount, 0.95, 10)").alias("p95"), 5
                        )
                )
        )
```

9.3. Análisis comparativo de latencias por características de servidores

Con el objetivo de minimizar latencias a la hora de analizar las operaciones realizadas en las criptomonedas seleccionadas, se ha decidido desplegar esta funcionalidad en la region Japan East de Azure. Con ello teóricamente deberemos conseguir una reducción de la latencia. Por otra parte, la máquina on premise en la que lanzamos el análisis, es bastante más potente que el contenedor que vamos a desplegar en Azure Container Instances, así que es otro factor a tener en cuenta. Para comprobarlo hemos lanzado el script en ambas máquinas y hemos medido la latencia en una muestra de 60 señales, obteniendo los siguientes resultados:

Concepto	Local (Canarias)	Japan East	Reducción de latencia media	Diferencia (%)

Latencia en recibir el mensaje	268,7 ms	109,5 ms	159,2 ms	-59,3%
Tiempo de procesado	6,9 ms	5.0 ms	1,9 ms	-27,5%
Total	**275,8 ms**	**112,7 ms**	**163,1 ms**	-59.1%

Podemos observar que:

- La latencia en la recepción del mensaje se ha reducido un 59,3% (159,2 ms) de media.

- También se ha reducido mínimamente el tiempo de procesado: -1.9 ms (-27.5%)

9.4. Fuentes de datos en profundidad

9.4.1. Fuentes de datos API REST

- **Binance:**

  Exchange de criptomonedas SPOT más grande del mundo por volumen de transacciones. Provee información vía API y vía Websocket.

  - **Binance SPOT**

    - **URL base:**

      - https://api.binance.com/api/v3/

- **Endpoints relativos:**
  - exchangeInfo: información relativa a las criptomonedas disponibles para operar en el mercado spot. No incluye datos de periódicos.

- **Binance Futures**

Exchange de criptomonedas Futures más grande del mundo por volumen de transacciones. Provee información vía API

- **URL base:**
  - https://fapi.binance.com/fapi/v1/

- **Endpoints relativos:**
  - exchangeInfo: Detalles de los mercados disponibles: símbolos, filtros, precisiones, etc. No incluye datos de periódicos.
  - aggTrades**:** Histórico de operaciones agregadas (trades consolidados por precio/cantidad) con periodicidad de 1 segundo.
  - Klines: Velas OHLCV (Open, High, Low, Close, Volume) por símbolo y periodo.
  - fundingRate: Historial de tasas de financiación aplicadas a contratos perpetuos.
  - openInterest: Nivel actual de contratos abiertos (open interest) por símbolo.

- premiumIndex: Mark price, índice de referencia y funding.
- forceOrders: Historial de liquidaciones forzadas (margin calls).

- **Bybit**

  Exchange global de criptomonedas que ofrece trading de spot y derivados. Provee información vía API.

  - **URL base:**
    - https://api.bybit.com/v5/market/
  - **Endpoints:**
    - instruments-info?category=spot: Devuelve la lista de contratos de futuros lineales disponibles, con su configuración detallada (símbolo, tamaño mínimo, apalancamiento, etc.).
    - instruments-info?category=linear: Devuelve la lista de pares de trading en el mercado spot, junto con sus parámetros (símbolo, filtros de precio y cantidad, etc.).

- **OKX**
  - Información de criptomonedas en SPOT y futuros:
    - **URL base:** https://www.okx.com/api/v5/public/

- Endpoints:
  - instruments?instType=SPOT: Devuelve la lista de instrumentos del mercado spot disponibles en OKX, incluyendo símbolo, moneda base y cotizada, tamaño mínimo, y precisión.
  - instruments?instType=SWAP: Devuelve la lista de contratos perpetuos (swap) disponibles, con detalles como símbolo, tipo de margen, tamaño mínimo de contrato y moneda de liquidación.

- **CoinGecko**

Agregador de datos independiente de criptomonedas más grande del mundo. No permite realizar operaciones, solo provee información

- **Base URL:**
  - https://api.coingecko.com
- **Endpoints relativos:**
  - /api/v3/coins/{id}: Usado para obtener descripción, fechas, redes sociales, etc. de cada criptomoneda.

9.4.2. Fuentes de datos Websocket

- **Binance Websocket**

  Permite suscribirte a flujos de datos en tiempo real de la plataforma de Binance, como precios de activos, operaciones, velas y datos de cuenta, para automatizar estrategias de trading o acceder a información

  - **Base URL:**

    - wss://fstream.binance.com/

  - **Endpoint relativo generado dinámicamente:**

    - stream?streams={symbol}@aggTrade/{symbol}@bookTicker: envía información de operaciones ejecutadas agrupadas por precio y dirección, información en tiempo real del mejor precio de compra y venta y sus cantidades.

9.5. Proceso de trabajo y fases (ampliación)

El proyecto ha sido llevado a cabo siguiendo las siguientes fases

Fase 1: Evaluación y elección de los datos necesarios para analizar las criptomonedas candidatas a estudio en profundidad

Se analiza qué datos serán necesarios para analizar las distintas criptomonedas y concluir sobre su idoneidad para ser analizadas más en profundidad. Al ser un contexto tan complejo, será necesario hacer un estudio más en detalle para llegar a esas conclusiones.

- Duración: 10 horas.

- Dependencias: No.

- Entregables: relación de datos a estudiar.

Fase 2: Análisis de fuentes de datos

En esta fase se identifican y evalúan las fuentes de datos necesarias decididas en la fase anterior. El objetivo principal es determinar si es accesible, en qué formato se expone y con qué frecuencia puede obtenerse. Se intentará recabar datos de la mayor cantidad de criptomonedas posible, de fuentes fiables.

- Duración: 10 horas.

- Dependencias: No.

- Entregables: relación de fuentes de datos.

Fase 3: Evaluación y elección de tecnologías

En esta fase se seleccionan las tecnologías específicas que compondrán la arquitectura del proyecto. Aunque no se ha definido aún un *stack* concreto, se parte de los requisitos funcionales identificados: ingesta y procesamiento *batch* de datos históricos, almacenamiento eficiente de series temporales, y capacidad de escalar hacia una arquitectura en tiempo real en fases posteriores.

- Duración: 5 horas.

- Dependencias: No.

- Entregables: relación de tecnologías a utilizar

Fase 4: Ingesta y almacenamiento de datos

En esta fase se implementan los procesos de ingesta que permiten capturar los datos de las fuentes de datos seleccionadas en la fase 2, usando las tecnologías que mejor se adapten, decidido en la fase 3.

- Duración: 30 horas.
- Dependencias:
    - Fase 1, fase 2 y fase 3
- Entregables: datos almacenados en ADLS2 para su posterior procesamiento

Fase 5: Limpieza y análisis de los datos

En esta fase se lleva a cabo el saneamiento de los datos, comprobando su calidad, eliminando redundancias y, aunque se intentará evitar, se realizará la imputación de valores faltantes. A posteriori se analizarán los datos y se calcularán las métricas apropiadas para la elección de la criptomoneda más interesante para profundizar en su estudio.

- Duración: 30 horas.
- Dependencias:
    - Fase 4
- Entregables: datos saneados y métricas calculadas, almacenados en ADLS2.

Fase 6: Elección de la criptomoneda más interesante

Tras la fase anterior, la elección de la criptomoneda más interesante será un proceso casi trivial.

- Duración: 2 horas.

- Dependencias:

    - Fase 5

- Entregables: relación de 3 criptomonedas más interesantes.

Fase 7: Ingesta y almacenamiento de datos de las criptomonedas elegidas

Una vez seleccionadas las criptomonedas más relevantes en la fase anterior, se procede a la ingesta sistemática de sus datos históricos. En esta etapa se recopilan los valores de mercado y una muestra representativa de las transacciones realizadas en el mismo periodo. Se fija la periodicidad de cada fuente y el horizonte temporal a analizar.

-Duración: 30h

- Dependencias: fase 6

- Output: métricas almacenadas en ADLS2

Fase 8: limpieza y análisis de los datos

En esta fase se analizan en profundidad los datos recopilados de las criptomonedas seleccionadas para proceder a la limpieza de los mismos y generar un informe en el que poder ver los datos agregados y sacar estadísticas relevantes.

-Duración: 15h

- Dependencias: fase 7

- Output: métricas procesadas, combinadas y almacenadas en ADLS2 para su análisis manual.

Fase 9: construcción del prototipo de trading en tiempo real

En esta fase se desarrolla un prototipo funcional que simula un sistema de toma de decisiones de inversión en tiempo real, basado en las correlaciones identificadas en

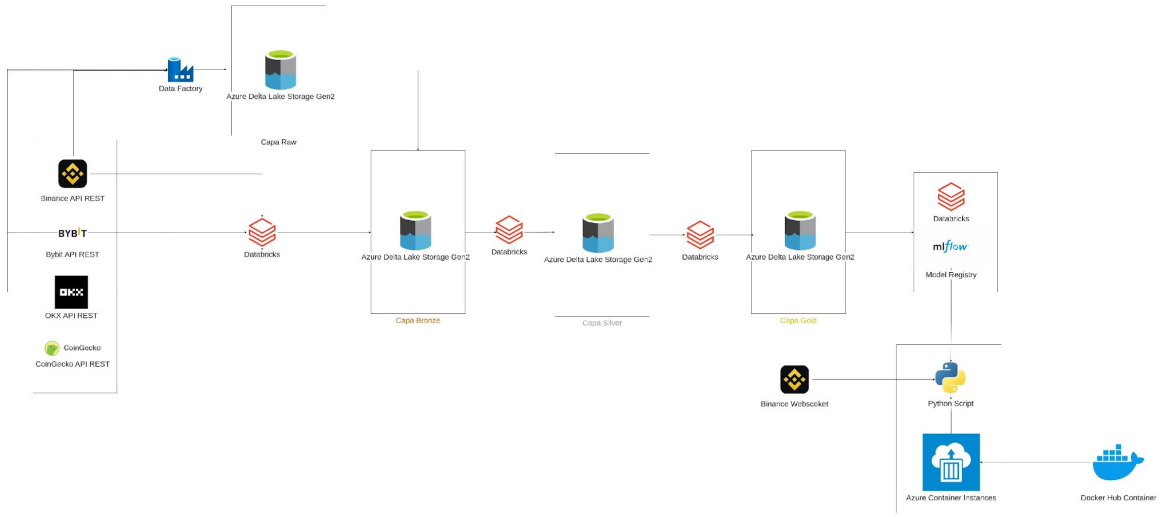
fases anteriores. El objetivo del mismo es reaccionar correctamente y con la mínima latencia posible, ante eventos detectados en la blockchain (como grandes transacciones) o en los mercados (como picos de volumen o volatilidad), ejecutando señales de compra o venta simuladas y observando la latencia observada. Para la implementación se utilizará Azure.

Fase 10: análisis de los datos en base a la latencia obtenida

En esta fase se evalúa el rendimiento temporal del sistema, analizando la latencia total desde que ocurre un evento relevante (como una gran transacción o cambio de volumen) hasta que el prototipo genera una señal de acción. Este análisis permite valorar la viabilidad de aplicar el sistema en entornos de *trading* automatizado o alerta temprana, utilizando los datos recabados en fases anteriores.

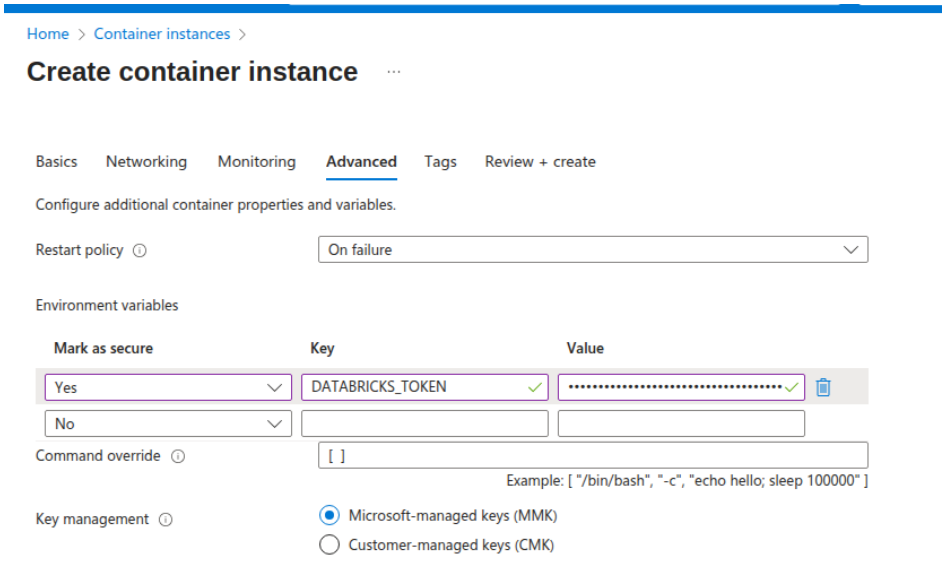
9.6. Descripción de la arquitectura técnica (ampliación)

9.7. Diagrama General (archivo de imagen adjunto en el entregable)

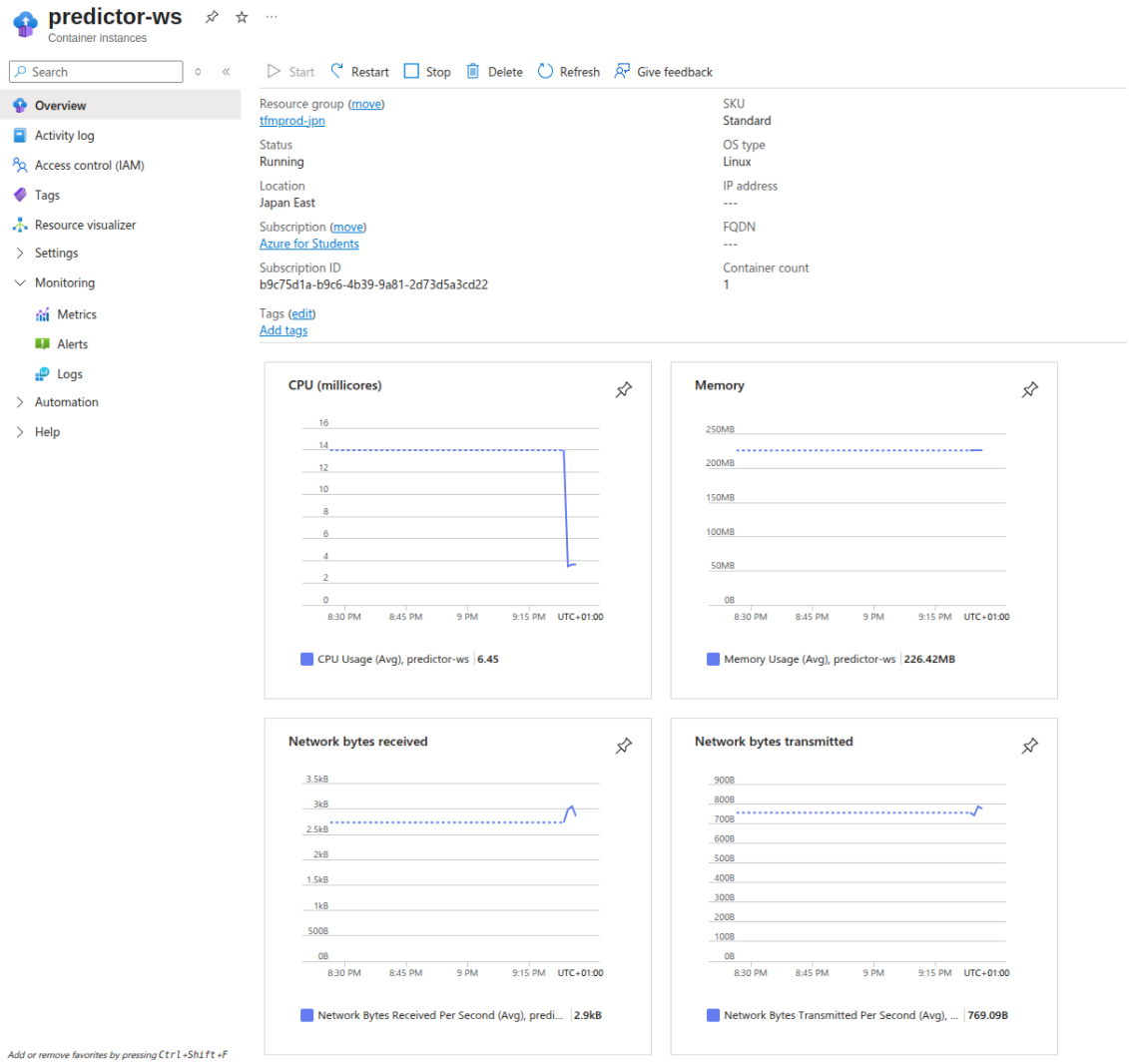


9.8. Creación y desempeño de contenedor Azure Container Instances

Capturas de pantalla del proceso de creación del contenedor para producción en Azure Container Instances:




Home > predictor-ws
predictor-ws | Containers
Container instances
Search
Refresh
Give feedback
Overview
Activity log
Access control (IAM)
Tags
Resource visualizer
Settings
Containers
Identity
Properties
Locks
Monitoring
Automation
Help
Events
Properties
Logs
Connect
To troubleshoot your container further, please navigate to Logs. Learn more
[mlflow] tracking_uri = databricks
[mlflow] registry_uri = databricks-uc
[boot] symbols = ['chrusdt', 'aliceusdt', 'dentusdt']
[boot] min_notional = {'chrusdt': 10385.81208, 'aliceusdt': 5881.16548, 'dentusdt': 13028.784481}
[ws] connecting wss://fstream.binance.com/stream?streams=chrusdt@aggTrade/chrusdt@bookTicker/aliceusdt@aggTrade/aliceusdt@bookTicker/dentusdt@aggTrade/dentusdt@bookTicker
[ws] connected
{
"end_to_end_time_ms": 26,
"network_latency_ms": 11,
"notional": 12979.3482,
"prediction": 0.0,
"processed_at": 1758190452218,
"processed_at_str": "2025-09-18 10:14:12.218000",
"processing_latency_ms": 15,
"symbol": "ALICEUSDT",
"ts_msg": 1758190452192,
"ts_msg_str": "2025-09-18 10:14:12.192000"
}
{
"end_to_end_time_ms": 163,
"network_latency_ms": 157,
"notional": 9830.1996,
"prediction": 0.0,
"processed_at": 1758190591435,
"processed_at_str": "2025-09-18 10:16:31.435000",
"processing_latency_ms": 6,
"symbol": "ALICEUSDT",
"ts_msg": 1758190591272,
"ts_msg_str": "2025-09-18 10:16:31.272000"
}
{
"end_to_end_time_ms": 24,
"network_latency_ms": 18,
"notional": 9145.9368,
"prediction": 0.0,
"processed_at": 1758191268253,
"processed_at_str": "2025-09-18 10:27:48.253000",
"processing_latency_ms": 6,
"symbol": "ALICEUSDT",
"ts_msg": 1758191268229,
"ts_msg_str": "2025-09-18 10:27:48.229000"


predictor-ws
Container instances
Search
Overview
Activity log
Access control (IAM)
Tags
Resource visualizer
Settings
Monitoring
Metrics
Alerts
Logs
Automation
Help
Start
Restart
Stop
Delete
Refresh
Give feedback
Resource group (move)
tfmprod-jpn
Status
Running
Location
Japan East
Subscription (move)
Azure for Students
Subscription ID
b9c75d1a-b9c6-4b39-9a81-2d73d5a3cd22
Tags (edit)
Add tags
SKU
Standard
OS type
Linux
IP address

FQDN

Container count
1
CPU (millicores)
CPU Usage (Avg), predictor-ws 6.45
Memory
Memory Usage (Avg), predictor-ws 226.42MB
Network bytes received
Network Bytes Received Per Second (Avg), predi... 2.9kB
Network bytes transmitted
Network Bytes Transmitted Per Second (Avg), ... 769.09B
Add or remove favorites by pressing Ctrl+Shift+F

9.9. Relación de tablas creadas en Unity Catalog

BRONZE

dbrtfmprod.whale_trades.bronze__top_kyles_all_trades

- Qué guarda: aggTrades de Binance Futures, una fila por trade agregado.
- Campos: symbol, agg_trade_id, price, qty, first_trade_id, last_trade_id, timestamp (ms), ts (timestamp), sec (segundo), date, trade_value. Particionado: symbol.
- Notebook: 01-ingesta_aggtrades

dbrtfmprod.whale_trades.bronze__{exchange}_exchangeinfo_futures

- Qué guarda: snapshots RAW→Bronze de exchange info de mercados futuros (por exchange = binance|bybit|okx).
- Campos: metadatos del mercado (tamaños de lote/paso, precisiones, estado, coin base/quote/settlement, límites).
- Particionado: _ingested_filename.
- Notebook: exchangeinfo_raw2bronze_futures_and_spot

dbrtfmprod.whale_trades.bronze__{exchange}_exchangeinfo

- Qué guarda: snapshots RAW→Bronze de exchange info de spot (por exchange).
- Campos típicos: metadatos de cada par spot (ticks, tamaños, estado, precisiones).
- Particionado: _ingested_filename.
- Notebook: exchangeinfo_raw2bronze_futures_and_spot

SILVER

dbrtfmprod.whale_trades.silver__base_1s_trades

- Qué guarda: base temporal a 1 segundo por símbolo, con precio continuo y agregados por segundo.
- Particionado: symbol.
- Campos: symbol, ts, mid (último precio conocido), last_price_1s, vol_base_1s, vol_quote_1s, n_trades_1s, buy_aggr_qty_1s, sell_aggr_qty_1s, ofi_trades_1s.
- Notebook: 02-base_1s_trades

dbrtfmprod.whale_trades.silver__features_1s

- Qué guarda: features derivadas sobre la base 1s (retornos, volatilidad, flujos, rachas, actividad).
- Particionado: exchange.
- Notebook: 03_features

dbrtfmprod.whale_trades.silver__labels_1s

- Qué guarda: etiquetas a futuro por varios horizontes H (segundos).
- Campos: mid_fwd_{H}s, r_fwd_{H}s, y{H} (triclase), y{H}_up (binaria), más symbol, date, ts.
- Particionado: —

- Notebook: 04-labels

dbrtfmprod.whale_trades.silver__binance_exchangeinfo_futures

- Qué guarda: exchange info de Binance futuros ya normalizada (una fila por símbolo futuro).
- Campos: symbol, base/quote/settlement coin, status, max_leverage, tick_size, step_size, min/max_order_qty, price/quantity/base/quotePrecision, filtros/flags, ...
- Particionado: symbol
- Notebook: binance_crypto_bronze2silver

dbrtfmprod.whale_trades.silver__criptolist_futures

- Qué guarda: lista consolidada de pares futuros válidos por exchange.
- Campos: symbol, base_coin, quote_coin, settlement_coin, status, max_leverage, tick_size, step_size, min_order_qty, max_order_qty, listed_at, _ingested_at, processed_at, exchange.
- Particionado: symbol.
- Notebook: exchangeinfo_bronze2silver_futures_and_spot

dbrtfmprod.whale_trades.silver__criptolist_spot

- Qué guarda: lista consolidada de pares spot válidos por exchange.

- Campos: symbol, base_coin, quote_coin, status, min_order_qty, max_order_qty, tick_size, _ingested_at, processed_at, exchange.
- Particionado: _ingested_at.
- Notebook: exchangeinfo_bronze2silver_futures_and_spot.py.

GOLD

dbrtfmprod.whale_trades.gold__dataset_1s

- Qué guarda: dataset de entrenamiento (join 1:1 features + labels por symbol,date,ts) sin duplicados.
- Particionado: symbol
- Notebook: 04-labels

dbrtfmprod.whale_trades.gold__kyles_top

- Qué guarda: ranking diario de símbolos con score_final y métricas OHLCV diarias.
- Campos: symbol, score_final, open, high, low, close, volume_base, volume_quote, trades, takerBuyBase, takerBuyQuote, date.
- Particionado: --
- Notebook: binance_calculate_scores

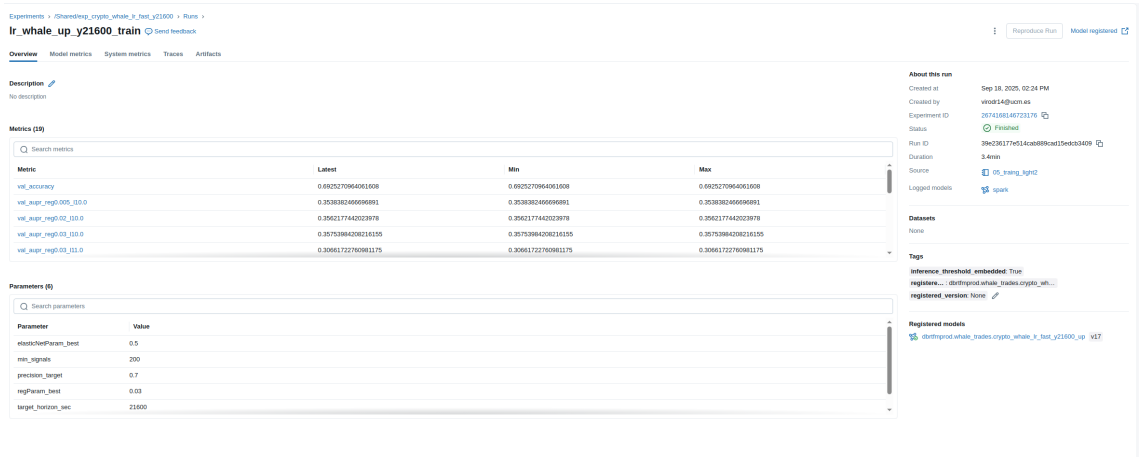
dbrtfmprod.whale_trades.gold__criptolist_current

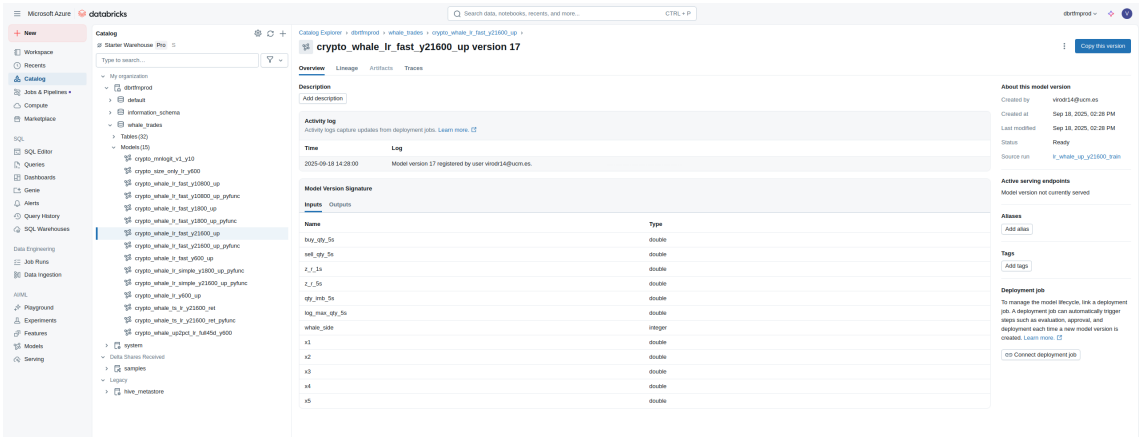
- Qué guarda: vista actual unificada (spot+futuros) de pares que pasan controles de calidad.
- Campos: symbol, base_coin, quote_coin, settlement_coin, status, etc., exchange, market_type, processed_at.
- Particionado: exchange, market_type.
- Notebook: exchangeinfo_silver2gold_futures_and_spot.py.

dbrtfmprod.whale_trades.gold__criptolist_quarantine

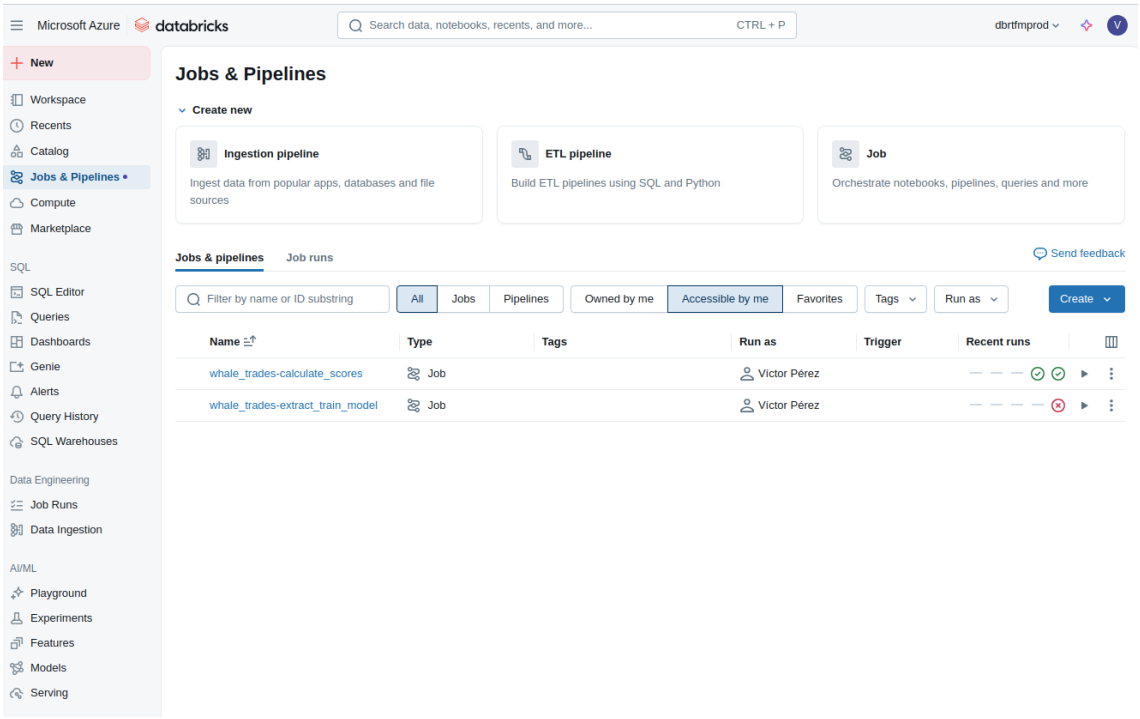
- Qué guarda: pares que no pasan DQ, con motivo y payload original para auditoría.
- Campos: reason, exchange, market_type, payload (JSON), processed_at.
- Particionado: exchange, market_type.
- Notebook: exchangeinfo_silver2gold_futures_and_spot.py.

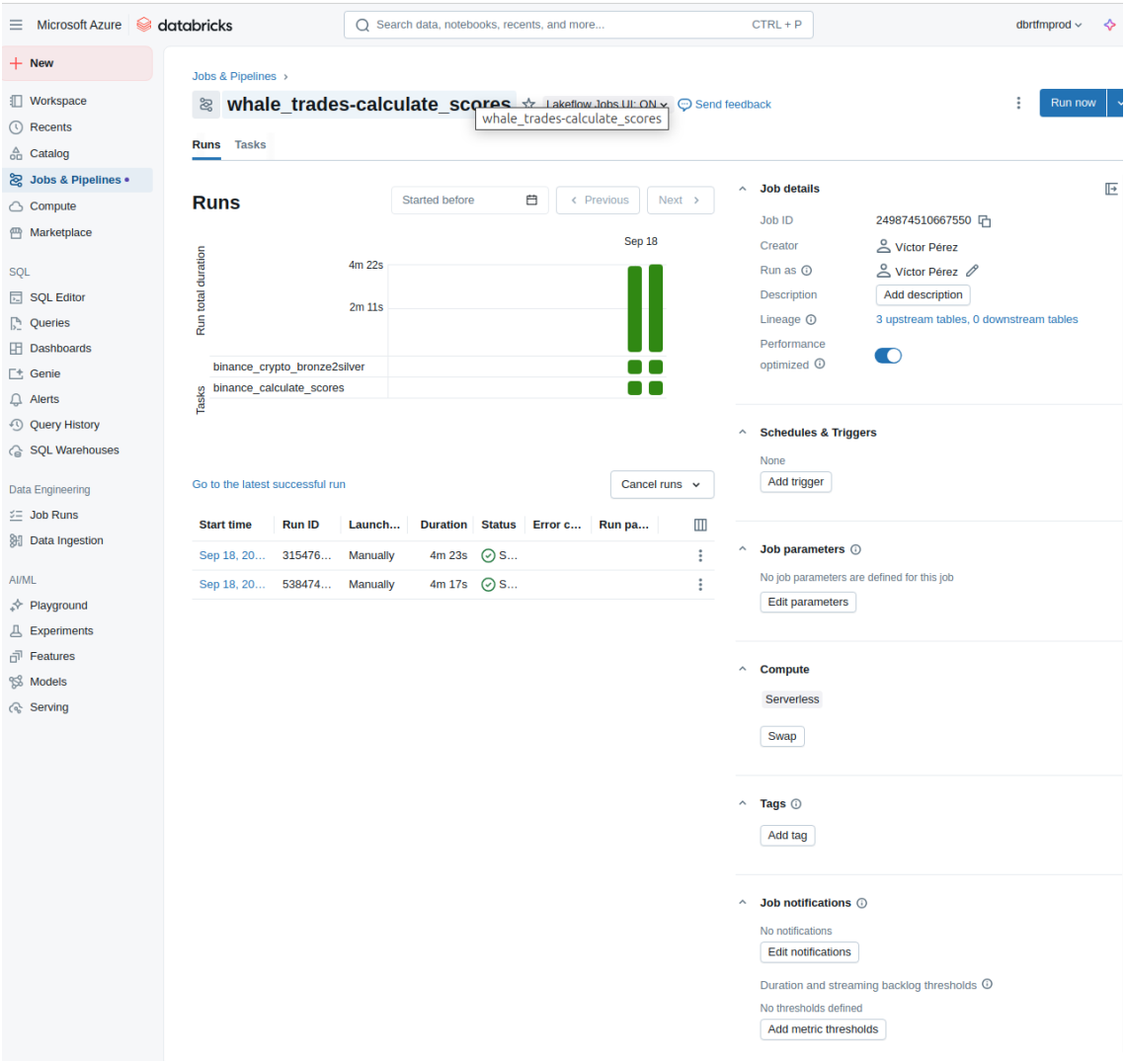
9.10. MLflow + Unity Catalog

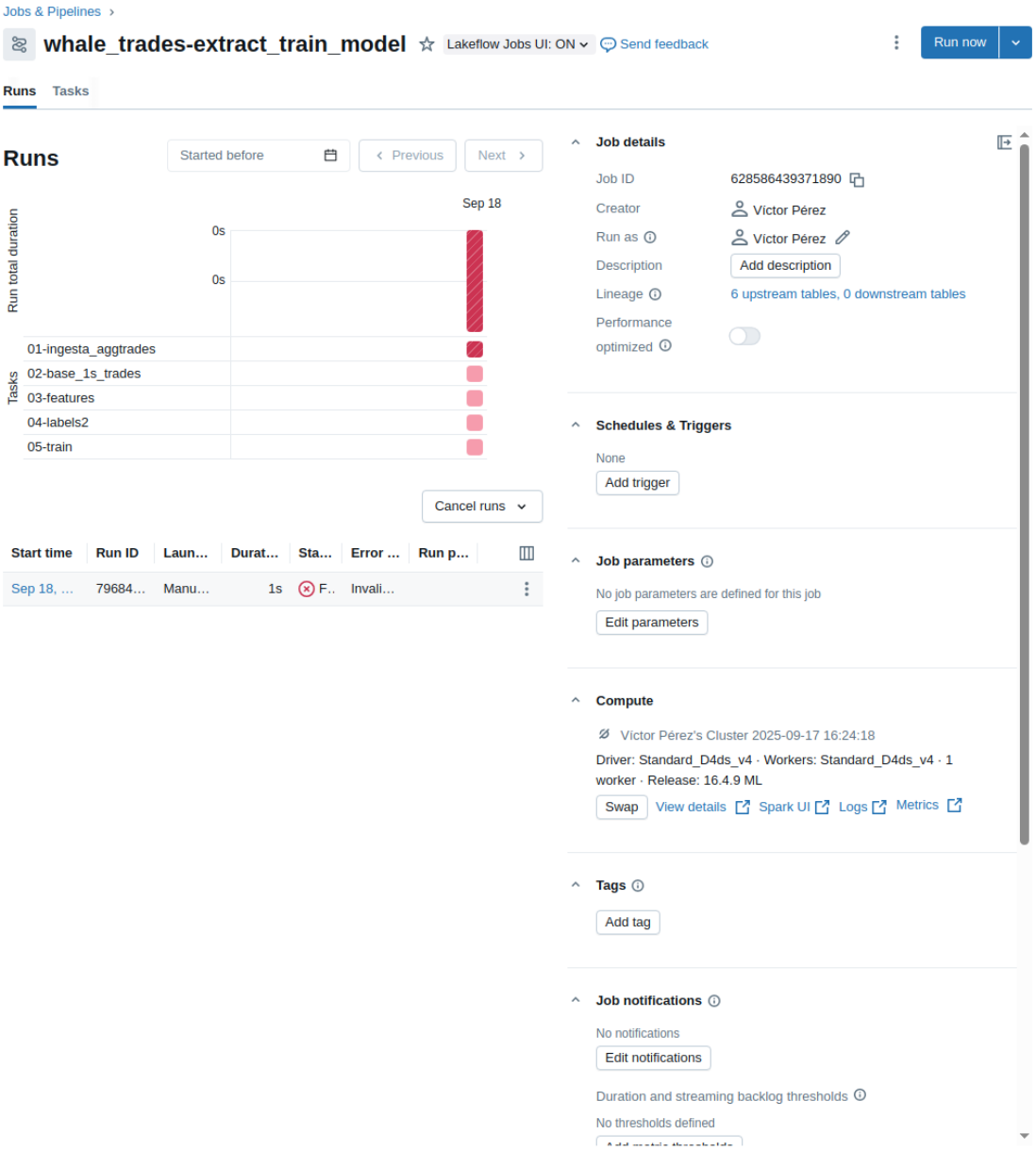




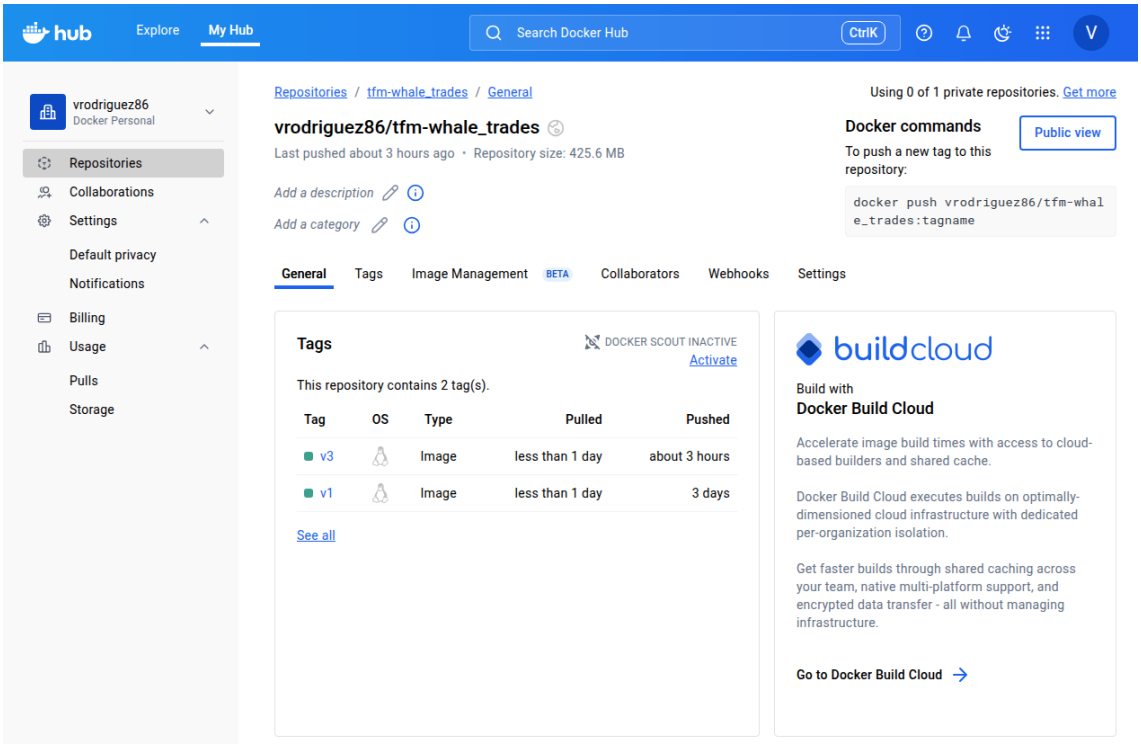
9.11. Databricks Jobs



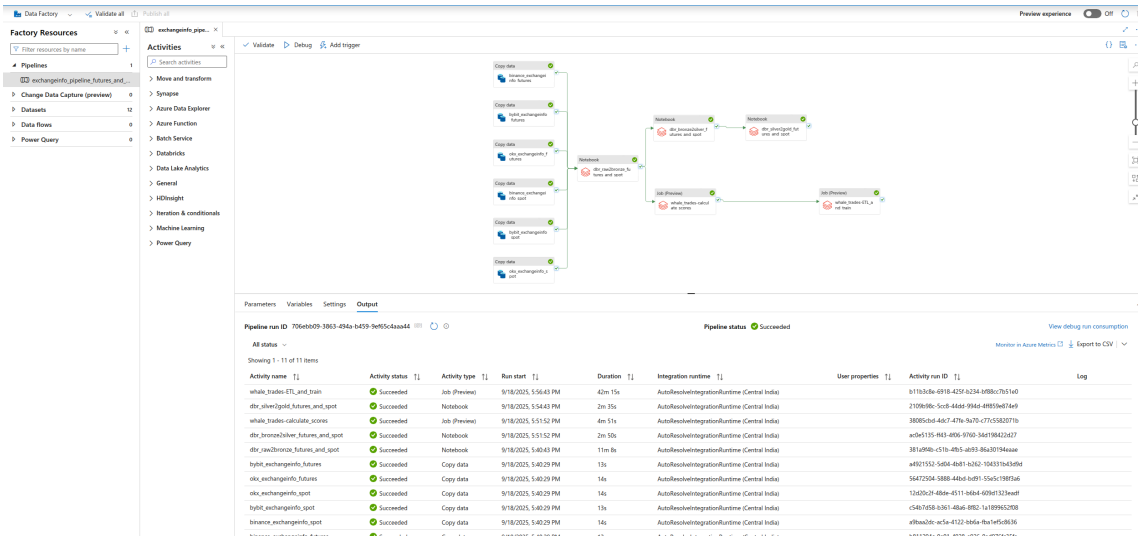

Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbrtfmprod
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Models
Serving
Jobs & Pipelines
whale_trades-calculate_scores
Lakeflow Jobs UI: ON
Send feedback
Run now
Runs
Tasks
Started before
Previous
Next
Sep 18
4m 22s
2m 11s
Run total duration
binance_crypto_bronze2silver
binance_calculate_scores
Tasks
Go to the latest successful run
Cancel runs
Start time
Run ID
Launch...
Duration
Status
Error c...
Run pa...
Sep 18, 20...
315476...
Manually
4m 23s
S...
Sep 18, 20...
538474...
Manually
4m 17s
S...
Job details
Job ID
249874510667550
Creator
Víctor Pérez
Run as
Víctor Pérez
Description
Add description
Lineage
3 upstream tables, 0 downstream tables
Performance optimized
Schedules & Triggers
None
Add trigger
Job parameters
No job parameters are defined for this job
Edit parameters
Compute
Serverless
Swap
Tags
Add tag
Job notifications
No notifications
Edit notifications
Duration and streaming backlog thresholds
No thresholds defined
Add metric thresholds

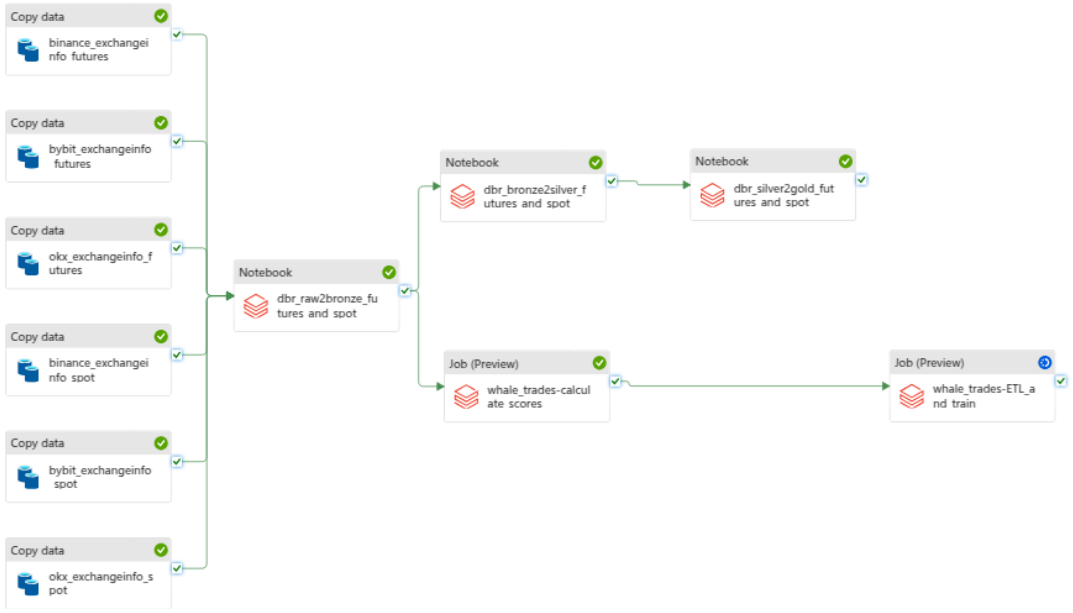

Jobs & Pipelines ›
whale_trades-extract_train_model
Lakeflow Jobs UI: ON
Send feedback
Run now
Runs
Tasks
Runs
Started before
Previous
Next
Sep 18
Run total duration
0s
0s
Tasks
01-ingesta_aggtrades
02-base_1s_trades
03-features
04-labels2
05-train
Cancel runs
Start time
Run ID
Laun…
Durat…
Sta…
Error …
Run p…
Sep 18, …
79684…
Manu…
1s
F..
Invali…
Job details
Job ID
628586439371890
Creator
Víctor Pérez
Run as
Víctor Pérez
Description
Add description
Lineage
6 upstream tables, 0 downstream tables
Performance optimized
Schedules & Triggers
None
Add trigger
Job parameters
No job parameters are defined for this job
Edit parameters
Compute
Víctor Pérez's Cluster 2025-09-17 16:24:18
Driver: Standard_D4ds_v4 · Workers: Standard_D4ds_v4 · 1 worker · Release: 16.4.9 ML
Swap
View details
Spark UI
Logs
Metrics
Tags
Add tag
Job notifications
No notifications
Edit notifications
Duration and streaming backlog thresholds
No thresholds defined

9.12. Docker Hub



8.13. Azure Data Factory





8.14 Azure Container Instances

